



Inter-process Communication

The Need for Various Types of IPC Objects

In this article, we explore the basic inter-process communication (IPC) techniques that are available.

A couple of questions: What is IPC? And do normal users require it often?

IPC occurs when processes exchange data through some communication channel. For a general-purpose OS, processes do not access the memory of other processes due to protection mechanisms. So IPC is essential for when a process needs data from another process.

Implicitly, we may use IPC every day. Browsing the WWW is an example. Here, the processes involved are the local browser and the remote Web server. The communication channel is the Internet, and the protocol is HTTP. Another example is if we run the 'command pipeline' `who | wc -l` to print the number of users logged in on the system. This also is an IPC technique. Thus, we often use IPC techniques when we work.

Broadly, we can classify common or widely available IPC techniques in three groups: pipes and FIFO; message queues, shared memory and semaphores; and sockets. The first group has *process-persistent* IPC techniques—these IPC objects are available while the processes that use them are running. The second group of IPC objects is *kernel-persistent*; they are

available till the system reboots, or they are explicitly deleted. The third group of IPC objects can help us communicate over a network. Groups 1 and 2 are generally used to communicate between processes on a single computer.

In the subsequent sections, we will look at algorithms and C code to use some of these IPC objects to implement a generic sender and receiver. I have provided the algorithms as pseudo code, which you can use for reference, to improve the code. I mainly focus on the core concepts and avoid explaining too many syntactic details, which you can learn from the Linux manual pages.



Note: Source code is omitted for brevity; please download it from http://www.linuxforu.com/article_source_code/sept12/inter_process_communication.zip.

Pipes and FIFO

In the previous section's example, the `who|wc -l` command used a communication channel called a *pipe*. A pipe has two ends (represented by descriptors)—*read* and the *write* end.